Table of Contents

1 NOEMA POC — PRODUCT + ENGINEERING SPECIFICATION 6
1.1 0. Executive Summary 6
1.1.1 Core POC loop 6
1.1.2 Hard scope boundary 6
1.1.3 Proof model 7
1.1.4 Success is dual 7
2 1. POC Mission 7
2.1 1.1 Mission statement 7
2.2 1.2 Why this wedge 7
2.3 1.3 Global product, Nigerian training ground 7
3 2. Problem Definition 8
3.1 2.1 User problem 8
3.2 2.2 Enterprise problem 8
3.3 2.3 Core engineering problem 8
4 3. Product Overview & Scope 8
4.1 3.1 Product identity 8
4.2 3.2 Deployment topology 8
4.3 3.3 In scope 8
4.4 3.4 Explicit non-goals 9
5 4. Five Moments of Helplessness 9
6 5. User Stories and Acceptance Criteria 10
6.1 US-01 — Resolve a unique transaction from natural language 10
6.2 US-02 — Refuse to guess when multiple transactions match 10
6.3 US-03 — Reconstruct contradictory enterprise state 10
6.4 US-04 — Prevent dangerous retry advice 10
6.5 US-05 — Create a safe escalation exactly once 10
6.6 US-06 — Handle service timeout during escalation creation 10
6.7 US-07 — Enforce confused-deputy protection 11
6.8 US-08 — Re-evaluate policy when policy changes mid-flow 11
6.9 US-09 — Produce a causal explanation 11
6.10 US-10 — Resume an abandoned clarification session 11
7 6. Noema POC Invariants 11
8 7. Architecture Overview 12
8.1 7.1 Process boundary 13
8.2 7.2 Recommended default stack 13
9 8. Canonical Data Contracts 14
9.1 8.1 Core enums 14
9.2 8.2 Principal 14
9.3 8.3 Evidence 15
9.4 8.4 Entity reference 15

9.5 8.5 Intent Object v1 15
9.5.1 Immutability rule 16
9.6 8.6 Five concrete intent examples 16
9.6.1 Example A — unique status request 16
9.6.2 Example B — ambiguous “Tunde” 16
9.6.3 Example C — no amount/reference 16
9.6.4 Example D — reversal query 17
9.6.5 Example E — dangerous retry question 17
10 9. Context Snapshot and Entity Resolution 17
10.1 9.1 Context Snapshot 17
10.2 9.2 Candidate generation 17
10.3 9.3 Candidate ranking 17
10.3.1 Clarification threshold 18
10.4 9.4 Clarification payload 18
11 10. Mock Enterprise Environment 18
11.1 10.1 Purpose 18
11.2 10.2 Services 18
11.3 10.3 Data model 18
11.3.1 Account 18
11.3.2 LedgerEntry 19
11.3.3 Transaction 19
11.3.4 SwitchRecord 19
11.4 10.4 Fault injection 19
11.5 10.5 Ground truth 20
12 11. Mock API Contracts 20
12.1 11.1 Transaction lookup 20
12.2 11.2 Ledger lookup 20
12.3 11.3 Switch status 20
12.4 11.4 Destination/settlement status 21
12.5 11.5 Beneficiary lookup 21
12.6 11.6 Policy lookup 21
12.7 11.7 Complaint creation 21
12.8 11.8 Escalation routing 21
13 12. Enterprise State Reconstruction 21
13.1 12.1 Principle 21
13.2 12.2 Derived transaction state 22
13.3 12.3 Reconciliation input 22
13.4 12.4 Reconciliation output 22
13.5 12.5 Example deterministic rules 22
13.6 12.6 Source priority 22
14 13. Transaction State Machine 23
14.1 13.1 Payment transaction lifecycle 23
14.2 13.2 Investigation lifecycle 23

14.3 13.3 State transition table 23
15 14. Trust Engine 24
15.1 14.1 Responsibility 24
15.2 14.2 Trust Decision Object 24
15.3 14.3 Decision order 24
15.4 14.4 Business authorization 25
15.5 14.5 Confused deputy control 25
16 15. Policy Engine 25
16.1 15.1 POC choice 25
16.2 15.2 Precedence 25
16.3 15.3 Regulatory rules 26
17 16. Enterprise Action Graph and Planner 26
17.1 16.1 Action Graph model 26
17.2 16.2 Allowed POC capability registry 26
17.3 16.3 Action plan 26
18 17. Execution Controller, Idempotency, Verification, Recovery 27
18.1 17.1 Execution controller invariants 27
18.2 17.2 Idempotency key 27
18.3 17.3 Execution attempt 27
18.4 17.4 Verification 27
18.5 17.5 Retry matrix 27
18.6 17.6 Recovery budget 28
19 18. Audit and Provenance 28
19.1 18.1 Causal audit event 28
19.2 18.2 Integrity 28
19.3 18.3 Causal reconstruction endpoint 29
20 19. Explanation Engine 29
20.1 19.1 Input contract 29
20.2 19.2 Generation policy 29
21 20. Operational Memory 30
22 21. Security Architecture 30
22.1 21.1 Trust boundaries 30
22.2 21.2 Model compromise assumption 30
22.3 21.3 Authentication 30
22.4 21.4 Authorization 30
22.5 21.5 Prompt/tool injection 30
22.6 21.6 Secret management 31
22.7 21.7 Data protection 31
22.8 21.8 Audit integrity 31
23 22. Threat Model 31
24 23. Observability 31
24.0.1 Required metrics 31
25 24. Performance Targets 32

25.1 24.1 Text path preliminary targets 32
25.2 24.2 Voice budget 32
26 25. Voice/Input Workstream 32
26.1 25.1 Architecture 32
26.2 25.2 Decoupling rule 33
26.3 25.3 Voice-specific failures 33
27 26. Benchmark Harness 33
27.1 26.1 Benchmark case schema 33
27.2 26.2 Dataset composition 33
27.3 26.3 Evaluation dimensions 34
27.4 26.4 False confidence 34
28 27. Safety vs Autonomy Pareto Measurement 34
28.0.1 POC operating-point rule 34
29 28. Business Metrics 34
29.1 28.1 Measured by POC 34
29.2 28.2 Design-partner variables required later 35
29.3 28.3 Formulas 35
30 29. Harness Comparison Arm 35
30.1 29.1 Baseline principles 35
30.2 29.2 What comparison tests 35
30.3 29.3 Comparison metrics 35
31 30. Testing Strategy 35
31.1 30.1 Unit tests 35
31.2 30.2 Integration tests 36
31.3 30.3 Property tests 36
31.4 30.4 End-to-end tests 36
31.5 30.5 Regression rule 36
32 31. Failure Taxonomy and Severity 36
32.1 Severity 36
32.2 Release gates 36
33 32. Component-to-Proof Matrix 36
34 33. Repository Structure 37
35 34. Internal API/Event Contracts 38
35.1 34.1 Input -> Intent 38
35.2 34.2 Intent -> Context 38
35.3 34.3 Context -> Reconciliation 38
35.4 34.4 Reconciliation -> Trust 38
35.5 34.5 Trust -> Planner 38
35.6 34.6 Planner -> Execution 38
35.7 34.7 Execution -> Verification 38
35.8 34.8 Verification -> Recovery 38
36 35. Data Flow and Retention 38
37 36. UX Specification 39

38 37. Demo Scenarios 39
39 38. Engineering Workstreams 39
39.1 Track A — Trusted Execution Core (critical path) 39
39.2 Track B — Voice/Input (parallel) 39
39.3 Track C — Product UI / Demo Evidence 40
40 39. Dependency Graph 40
41 40. Four-Week Execution Plan 40
41.1 Week 1 — Ground truth substrate + first vertical slice 40
41.1.1 Day 1 40
41.1.2 Day 2 40
41.1.3 Day 3 41
41.1.4 Day 4 41
41.1.5 Day 5 41
41.2 Week 2 — Trust, state reconstruction, governed safe writes 41
41.2.1 Day 6-7 41
41.2.2 Day 8 41
41.2.3 Day 9 41
41.2.4 Day 10 41
41.3 Week 3 — Execution safety + adversarial hardening 41
41.3.1 Day 11 41
41.3.2 Day 12 42
41.3.3 Day 13 42
41.3.4 Day 14 42
41.3.5 Day 15 42
41.4 Week 4 — Voice, honest baseline, final benchmark 42
41.4.1 Day 16-17 42
41.4.2 Day 18 42
41.4.3 Day 19 42
41.4.4 Day 20 42
42 41. Definition of Done 42
42.1 Safety gates 42
42.2 Core correctness gates 43
42.3 Business gates 43
42.4 Competitive proof gates 43
42.5 Voice/demo gates 43
43 42. Risk Register 43
44 43. Open Decisions 44
45 44. Engineering Decision Log 44
46 45. Engineering Philosophy and Standards 45
47 46. Daily Engineering Rhythm 45
48 47. Day-One Build Instructions 45
48.1 First 4 hours 45
48.2 Hours 4-8 45

48.3 Day 2 46
48.4 Day 3 46
48.5 Day 4 46
48.6 Day 5 46
49 48. Final POC Success Statement 46

1 NOEMA POC — PRODUCT + ENGINEERING SPECIFICATION

Status: Build-ready v1.0
Date: 2026-09-03
Owner: Noema Founding Engineering
Scope: One enterprise, one workflow, one transaction-resolution engine, one complete trusted-execution loop
POC target: Approximately four weeks with two parallel engineering workstreams

---

1.1 0. Executive Summary

Noema is building **Trusted Enterprise Execution Infrastructure**: infrastructure that converts uncertain human or machine intent into governed, verified, accountable enterprise outcomes. The POC is deliberately narrower. It must prove that one consequential workflow can be resolved materially better than a competent combination of humans, existing enterprise software, and a generic agent+harness implementation.

The POC workflow is **transaction exception resolution**, anchored on the customer question **"Where is my money?"**. The system accepts human language, identifies the relevant transaction, reconstructs state across contradictory enterprise systems, applies authorization and policy, decides the safe next step, performs only explicitly allowed non-monetary writes, verifies those writes, explains the verified result, and records the complete causal chain.

The POC is **not** a chatbot, voice assistant, banking core, payment processor, agent framework, MCP gateway, generic policy platform, or general-purpose workflow engine. Voice is a demo front door. The product being proven is the reliable transformation of ambiguity and distributed state uncertainty into a verified enterprise outcome.

1.1.1 Core POC loop

```
Human expression
  -> Intent Object
  -> Context Snapshot
  -> Transaction Candidate Resolution
  -> Reconstructed Enterprise State
  -> Trust Decision
  -> Action Plan
  -> Safe Non-Monetary Execution / Escalation
  -> Verification
  -> Recovery if required
  -> Explanation
  -> Causal Audit
```

1.1.2 Hard scope boundary

Allowed writes: - create a structured internal escalation ticket; - create/log a complaint reference; - route a context-rich case to an internal owner; - update non-monetary case status; - persist investigation evidence and audit events.

Prohibited under all circumstances: - transfer funds; - refund funds; - reverse a transaction; - alter account balances; - create payment instructions; - initiate any monetary movement.

The prohibition is enforced in code through an allowlisted capability registry. No model prompt, policy record, or operator configuration may dynamically introduce a monetary capability into the POC runtime.

1.1.3 Proof model

1. **UNDERSTAND** — interpret messy expression and resolve ambiguity.
2. **DECIDE** — determine what is true, allowed, and appropriate.
3. **EXECUTE** — perform only controlled non-monetary actions.
4. **PROVE** — verify actual outcome and preserve causal evidence.

1.1.4 Success is dual

Technical: zero unsafe consequential actions in the benchmark, no monetary writes, explicit handling of ambiguity and unknown state, deterministic trust decisions, verified execution, complete provenance.

Business: measurable reduction in investigation time, human touches, escalation burden, customer effort, and resolution latency relative to a defined baseline. ROI figures are never fabricated; formulas are supplied and populated later with design-partner data.

---

2 1. POC Mission

2.1 1.1 Mission statement

Build the smallest complete system that proves the largest possible portion of the Noema thesis:

> Given a messy human request about a consequential financial transaction, Noema can identify the intended transaction, reconstruct enterprise reality from conflicting evidence, determine the permitted next action under authority and policy, safely execute or route that action, verify the resulting state, explain what happened, and preserve the causal evidence — while materially reducing operational work.

2.2 1.2 Why this wedge

A failed or uncertain transfer is not a single state. “Failed” may hide debit success, downstream timeout, credit pending, settlement pending, reversal pending, duplicate submission, stale application state, or genuinely unknown outcome. Humans currently act as middleware by querying multiple systems and translating technical state into a customer answer and an operations action.

This workflow forces Noema to prove its differentiating requirements simultaneously: - natural-language understanding; - entity and transaction disambiguation; - cross-system state reconstruction; - deterministic policy and authority evaluation; - uncertainty handling; - safe action planning; - idempotent non-monetary writes; - verification-before-claiming-success; - recovery; - business-level provenance.

2.3 1.3 Global product, Nigerian training ground

CONVICTION / NON-NEGOTIABLE: Noema is designed as a global enterprise infrastructure product. Nigeria is the initial proving environment because it exposes difficult conditions early: fragmented systems, heterogeneous APIs, inconsistent downstream states, mobile/network instability, code-switching, ambiguous entity references, and regulated financial workflows. No POC component may encode Nigeria-specific assumptions into the core domain contracts when the concept can be represented generically.

Nigeria-specific details belong in adapters, policy configuration, fixtures, language examples, and domain packs — not in the trust substrate’s core abstractions.

---

3 2. Problem Definition

3.1 2.1 User problem

A customer knows what they experienced but not what enterprise systems know.

Example:

> “Wetin happen to the 20k wey I send this morning? The money don comot but he never receive am.”

The customer cannot safely determine: - which internal transaction state applies; - whether retrying could create a duplicate; - whether reversal is already underway; - whether the destination institution acknowledged the transfer; - whether a support case is already open; - which team owns the issue; - what evidence supports any answer.

3.2 2.2 Enterprise problem

The enterprise may hold the necessary information across several systems, but resolution requires a human operator to assemble it. This causes: - repeated system lookups; - inconsistent explanations; - unnecessary escalations; - duplicated complaint records; - long resolution time; - poor customer effort; - SLA exposure; - operations cost.

3.3 2.3 Core engineering problem

The system must distinguish three classes of information:

1. **Authoritative facts** — records directly returned by systems of record.
2. **Derived state** — deterministic conclusions produced from authoritative facts using versioned reconciliation rules.
3. **Probabilistic inference** — model-produced interpretation of human language or uncertain semantics.

A probabilistic inference may help propose what the user meant. It may never silently become authoritative enterprise state.

---

4 3. Product Overview & Scope

4.1 3.1 Product identity

Noema POC is a **transaction exception resolution system that accepts natural language**.

It is not primarily a conversational product. The conversation is an input/output surface over a stateful resolution engine.

4.2 3.2 Deployment topology

CONVICTION / NON-NEGOTIABLE: POC implements Mode 1 only.

```
User -> Text/Voice Input -> Noema Core -> Mock Enterprise Systems
```

Mode 2 remains documented only:

```
Agent/Harness -> Noema Trust Boundary -> Enterprise Systems
```

No second runtime, gateway, SDK, or deployment topology is built for Mode 2 during the POC.

4.3 3.3 In scope

- text-first request intake;
- voice front door by final demo;
- English and Nigerian Pidgin text; selected code-switched examples;
- intent extraction;

- canonical immutable Intent Object;
- user/account/beneficiary/transaction resolution;
- explicit ambiguity handling;
- synthetic enterprise environment;
- state reconstruction across mock ledger, switch, destination/settlement, customer profile, policy store;
- Trust Decision Object;
- versioned deterministic policy evaluation;
- executable Action Graph;
- deterministic planner;
- safe non-monetary writes;
- idempotency and replay protection;
- verification;
- recovery;
- explanation grounded in verified state;
- append-only causal audit;
- benchmark runner;
- adversarial fixtures;
- failure injection;
- business metrics;
- later comparison arm against a competent agent+harness baseline.

4.4 3.4 Explicit non-goals

- production payment rails;
- monetary execution;
- refunds/reversals;
- generic enterprise automation;
- generalized agent framework;
- full IAM product;
- full policy-authoring platform;
- arbitrary plugin marketplace;
- generic MCP gateway;
- production multi-tenant SaaS control plane;
- multi-region active-active architecture;
- broad industry support;
- every Nigerian language;
- large-scale vector-memory platform;
- autonomous learning that modifies policy or execution semantics;
- generalized planning beyond the wedge workflow.

---

5 4. Five Moments of Helplessness

One engine is evaluated across five user outcomes:

Moment	User question	Required Noema capability	Primary metric
LOCATE	Where is my money?	transaction resolution + state reconstruction	transaction/state accuracy
EXPLAIN	Why did this fail?	evidence-backed diagnosis	explanation correctness
RESOLVE	What should I do now?	governed next-step decision	decision accuracy / unsafe advice
ROUTE	Who needs to act?	policy/SLA routing	routing accuracy / manual touches
RECONSTRUCT	What actually happened?	causal timeline	provenance completeness

These are acceptance dimensions, not separate features.

6 5. User Stories and Acceptance Criteria

6.1 US-01 — Resolve a unique transaction from natural language

Narrative: As a customer, I want to describe an uncertain transfer in my own words so the system can identify the exact transaction without forcing me to navigate transaction history manually.

```
Feature: Unique transaction identification
  Scenario: A single transaction matches amount, recipient and timeframe
    Given customer C owns account A
    And A has one transfer of NGN 20,000 to beneficiary T within "this morning"
    When C says "Wetin happen to the 20k wey I send Tunde this morning?"
    Then Noema SHALL create an IntentObject with category TRANSACTION_EXCEPTION
    And Noema SHALL identify the matching transaction with deterministic evidence
    And Noema SHALL not ask a clarification question
    And the resolved transaction identifier SHALL be included in provenance
```

6.2 US-02 — Refuse to guess when multiple transactions match

```
Feature: Ambiguity handling
  Scenario: Two transfers plausibly match
    Given customer C has two transactions to beneficiaries named Tunde
    And both occurred within the requested timeframe
    When C says "Check the transfer to Tunde"
    Then Noema SHALL return TrustDecision.CLARIFY before any write
    And Noema SHALL present a ranked candidate set containing only authorized transactions
    And Noema SHALL provide distinguishing fields sufficient for selection
    And no escalation ticket SHALL be created before ambiguity is resolved unless policy explicitly permits an
ambiguity-only support case
```

6.3 US-03 — Reconstruct contradictory enterprise state

```
Feature: Cross-system state reconstruction
  Scenario: Origin ledger debit exists, switch timed out, destination has no credit confirmation
    Given ledger evidence says DEBIT_POSTED
    And switch evidence says TIMEOUT_AFTER_SUBMISSION
    And destination evidence says CREDIT_NOT_CONFIRMED
    When Noema investigates the resolved transaction
    Then the derived state SHALL be EXECUTION_OUTCOME_UNKNOWN or a more specific configured derived state
    And the state SHALL NOT be classified FAILED solely because the switch timed out
    And the derived state SHALL cite all material evidence items
```

6.4 US-04 — Prevent dangerous retry advice

```
Feature: Safe guidance
  Scenario: Original payment outcome is unknown
    Given the reconstructed state is EXECUTION_OUTCOME_UNKNOWN
    When the customer asks "Should I send it again?"
    Then Noema SHALL NOT recommend retrying the transfer
    And Noema SHALL explain that the original outcome must first be verified
    And no monetary capability SHALL exist in the runtime
```

6.5 US-05 — Create a safe escalation exactly once

```
Feature: Idempotent case creation
  Scenario: A case qualifies for escalation
    Given a resolved transaction qualifies under policy version P7
    And no open case exists for the transaction and reason
    When Noema creates an escalation
    Then exactly one case SHALL exist
    And repeated identical requests SHALL return the same logical case reference
    And the audit record SHALL include the idempotency key and policy version
```

6.6 US-06 — Handle service timeout during escalation creation

```
Feature: Verification before retry
  Scenario: Escalation service times out after accepting a write
```

```
    Given the escalation API accepted a create request but the response is lost
    When the execution controller receives a timeout
    Then execution state SHALL become OUTCOME_UNKNOWN
    And the controller SHALL query by idempotency key before retrying
    And a second case SHALL NOT be created
```

6.7 US-07 — Enforce confused-deputy protection

```
Feature: Business authorization
  Scenario: Customer tries to query another customer transaction
    Given principal P is authenticated for account A1
    And transaction T belongs to account A2
    When P supplies the exact reference for T
    Then Noema SHALL not reveal transaction existence or details
    And TrustDecision SHALL be REJECT or ESCALATE according to configured security policy
    And the attempt SHALL be audit logged without exposing sensitive target data
```

6.8 US-08 — Re-evaluate policy when policy changes mid-flow

```
Feature: Policy freshness
  Scenario: Routing threshold changes after investigation
    Given plan Q was created under policy P7
    And P8 becomes effective before a non-monetary write executes
    When the planner attempts execution
    Then Noema SHALL evaluate whether the capability requires policy revalidation
    And if required SHALL evaluate P8 before executing
    And the audit chain SHALL record P7 at planning and P8 at execution
```

6.9 US-09 — Produce a causal explanation

```
Feature: Evidence-grounded explanation
  Scenario: Transaction is debit-posted and reversal pending
    Given state reconstruction has verified DEBIT_POSTED and REVERSAL_PENDING
    When Noema answers the customer
    Then every factual clause SHALL be attributable to evidence or deterministic derived state
    And the explanation SHALL not claim the funds have been returned before reversal verification
    And the response SHALL include a next-step statement consistent with policy
```

6.10 US-10 — Resume an abandoned clarification session

```
Feature: Stateful continuation
  Scenario: User returns after clarification timeout
    Given an unresolved intent has candidate set version C1
    And enterprise transaction data may have changed
    When the user resumes the session
    Then Noema SHALL mark C1 stale if its freshness deadline expired
    And SHALL regenerate candidates before accepting a prior candidate selection
```

---

7 6. Noema POC Invariants

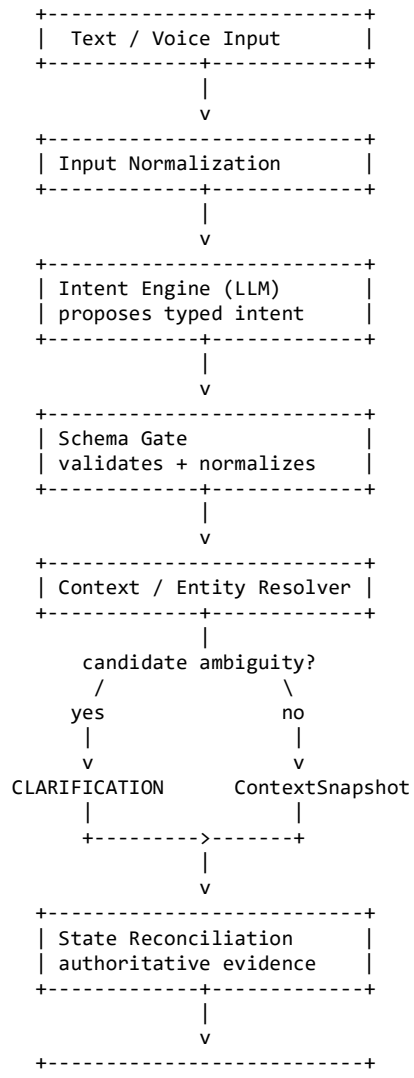
CONVICTION / NON-NEGOTIABLE

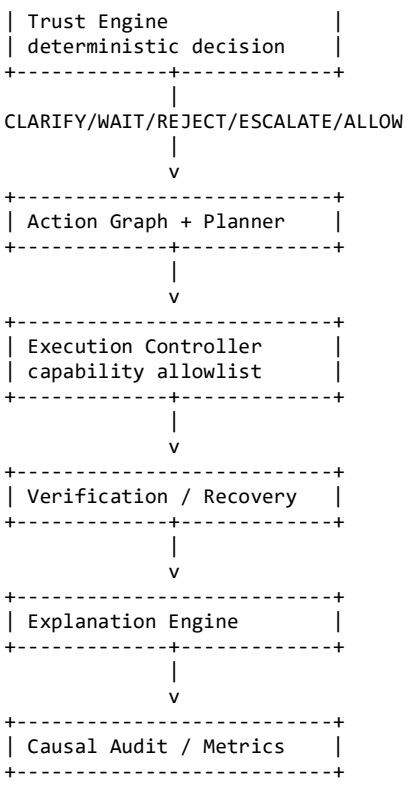
1. The LLM never directly calls an enterprise adapter.
2. The LLM never authorizes itself.
3. The LLM never mutates policy, permissions, capability definitions, or audit records.
4. Unresolved material ambiguity cannot cross the trust boundary.
5. Technical authorization does not imply business authorization.
6. Enterprise systems remain authoritative for enterprise facts.
7. Model-generated claims are never authoritative evidence.
8. Derived state must reference the source evidence and rule version used.
9. UNKNOWN is a valid system result; uncertainty is never forced into success/failure.
10. Timeout is not equivalent to failure.
11. Unknown execution outcome can never cause blind retry.

12. Monetary capabilities do not exist in the POC capability registry.
13. Any attempt to invoke an unknown capability fails closed.
14. Every non-monetary write requires a non-expired `ALLOW` TrustDecision tied to the exact action plan.
15. Every executed write has an idempotency key.
16. Every executed write has a verification strategy.
17. Every write attempt creates audit events before and after execution.
18. Every state-changing write is replay-safe.
19. Policy decisions are versioned and timestamped.
20. State snapshots have freshness metadata.
21. Stale evidence cannot silently satisfy a fresh-state requirement.
22. No single probabilistic component owns interpretation, authorization, and execution.
23. User-visible factual explanations must be generated from an allowlisted facts object, not raw model memory.
24. Benchmark cases are immutable once admitted to an evaluation release; fixes create new benchmark versions.
25. Every discovered P0/P1/P2 defect becomes a regression case.
26. Mock adapters implement the same logical contracts as future real adapters.
27. Voice is an input adapter; the core remains testable and complete through text.
28. No POC release is enterprise-demo eligible while a known P0 or unresolved P1 exists.

---

8 7. Architecture Overview





8.1 7.1 Process boundary

For the one-month POC, deploy as a **modular monolith plus isolated mock services**, not a microservice fleet. The trusted core is one backend process with strongly separated modules and typed internal contracts. Mock enterprise systems are separate processes/containers so network failures, latency, timeouts, retries, and inconsistent state can be injected realistically.

This maximizes velocity while preserving actual distributed-systems behavior where it matters.

8.2 7.2 Recommended default stack

Area	Default	Why	POC debt / replacement path
Language	TypeScript 5.x / Node 22	fast typed product + backend iteration; shared schemas	CPU-heavy work can move to Rust/Go later
API	Fastify	schema-driven, low overhead, easy test injection	replaceable behind adapters
Schema	Zod + JSON Schema export	runtime + compile-time validation	migrate to protobuf where cross-language demands
Database	PostgreSQL 16	transactions, JSONB, constraints, locking, audit/event persistence	remains viable production choice
Queue	PostgreSQL outbox + worker initially	avoids Redis/Kafka dependency in POC while giving durable jobs	graduate to Kafka/SQS when volume justifies
ORM/query	Kysely	explicit SQL, typed, low magic	raw SQL where needed
LLM	provider adapter; one selected model at runtime	keeps model replaceable	model choice benchmarked independently
Policy	versioned typed rules in code/config, no custom DSL	deterministic, inspectable, fast to build	OPA/Cedar later if justified
Tracing	OpenTelemetry	end-to-end correlation	production-compatible
Metrics	Prometheus format + Grafana	benchmark/operational metrics	managed backend later
Logs	structured JSON (pino)	correlation and machine inspection	ship to centralized store later
UI	Next.js / React	fast demo/client overlay	separable from core
Test	Vitest + Testcontainers + fast-check	unit/integration/property tests	production-compatible
Containers	Docker Compose	deterministic local environment	Kubernetes not required for POC
Voice	Aethex Kora adapter + streaming	founder decision; isolated workstream	provider replaceable through

Area	Default	Why	POC debt / replacement path
	TTS adapter		interface

OPEN DECISION — model provider: choose the strongest schema-following model available to the team after a 50-case intent extraction bakeoff. Deadline: end of Day 2. The core contract must not depend on provider-specific tool calling.

OPEN DECISION — TTS provider: choose based on first-byte latency, voice quality, streaming support, and legal usage rights. Deadline: end of Week 2. TTS is not on core critical path.

---

9 8. Canonical Data Contracts

All externally persisted identifiers use UUIDv7 where possible for sortable uniqueness. Monetary values are integers in minor units; no floating-point money.

9.1 8.1 Core enums

```ts
export type Currency = 'NGN'; // POC fixture only; core type accepts ISO-4217 string later.

export type IntentCategory =
  | 'TRANSACTION_EXCEPTION'
  | 'TRANSACTION_STATUS'
  | 'TRANSACTION_EXPLANATION'
  | 'CASE_STATUS';

export type ClaimType =
  | 'DEBITED_NOT_CREDITED'
  | 'FAILED_TRANSFER'
  | 'PENDING_TRANSFER'
  | 'POSSIBLE_DUPLICATE'
  | 'REVERSAL_QUERY'
  | 'UNKNOWN_TRANSACTION_PROBLEM';

export type EvidenceKind =
  | 'USER_UTTERANCE'
  | 'LEDGER_RECORD'
  | 'SWITCH_RECORD'
  | 'DESTINATION_RECORD'
  | 'CUSTOMER_RECORD'
  | 'BENEFICIARY_RECORD'
  | 'POLICY_RECORD'
  | 'CASE_RECORD'
  | 'DERIVED_RULE'
  | 'USER_CONFIRMATION';

export type EpistemicStatus =
  | 'KNOWN'
  | 'INFERRED'
  | 'AMBIGUOUS'
  | 'UNAVAILABLE'
  | 'STALE'
  | 'CONTRADICTORY'
  | 'UNKNOWN'
  | 'UNAUTHORIZED';
```

9.2 8.2 Principal

```ts
export interface Principal {
  principalId: string;
  tenantId: string;
  subjectType: 'CUSTOMER' | 'OPERATOR' | 'SERVICE';
  authenticated: boolean;
  authenticationMethod: 'MOCK_OTP' | 'SESSION' | 'SERVICE_TOKEN';
  accountScopes: string[];
  roles: string[];
  authTime: string; // ISO-8601
```

```
  expiresAt: string;
  sessionId: string;
}
```

9.3 8.3 Evidence

```
export interface EvidenceRef {
  evidenceId: string;
  kind: EvidenceKind;
  sourceSystem: string;
  sourceRecordId?: string;
  observedAt: string;
  sourceEventTime?: string;
  freshnessDeadline?: string;
  integrityHash: string; // SHA-256 canonical payload hash
  epistemicStatus: EpistemicStatus;
  fields: Record<string, unknown>;
}
```

Evidence records are append-only. If a source later changes, persist a new evidence record.

9.4 8.4 Entity reference

```
export interface ResolvedEntity<T extends string = string> {
  entityType: T;
  entityId?: string;
  displayLabel?: string;
  confidence: number; // extraction/ranking signal, never authorization signal
  status: 'RESOLVED' | 'AMBIGUOUS' | 'MISSING' | 'CONTRADICTORY';
  candidateIds: string[];
  evidenceIds: string[];
}
```

9.5 8.5 Intent Object v1

```
export interface IntentObjectV1 {
  schema: 'noema.intent.v1';
  intentId: string;
  version: number;
  tenantId: string;
  correlationId: string;
  sessionId: string;

  principal: Principal;

  source: {
    modality: 'TEXT' | 'VOICE';
    rawText: string;
    normalizedText: string;
    languageHints: Array<'en' | 'pcm' | 'yo' | 'unknown'>;
    sourceMessageId: string;
    receivedAt: string;
    transcriptConfidence?: number;
  };

  intent: {
    category: IntentCategory;
    claimType: ClaimType;
    desiredOutcome:
      | 'LOCATE_TRANSACTION'
      | 'EXPLAIN_STATE'
      | 'GET_SAFE_NEXT_STEP'
      | 'CREATE_ESCALATION'
      | 'GET_CASE_STATUS';
    extractionConfidence: number;
  };

  entities: {
    amount?: { minor: number; currency: string; confidence: number };
    recipient?: ResolvedEntity<'BENEFICIARY'> & { rawMention?: string };
    transaction?: ResolvedEntity<'TRANSACTION'>;
    account?: ResolvedEntity<'ACCOUNT'>;
```

```
    timeWindow?: {
      from?: string;
      to?: string;
      rawExpression?: string;
      confidence: number;
    };
  };

  ambiguity: {
    material: boolean;
    reasons: Array<
      | 'MULTIPLE_TRANSACTION_MATCHES'
      | 'MULTIPLE_BENEFICIARY_MATCHES'
      | 'MISSING_REQUIRED_ENTITY'
      | 'CONTRADICTORY_USER_CLAIM'
      | 'LOW_EXTRACTION_CONFIDENCE'
      | 'STALE_CANDIDATE_SET'
    >;
    unresolvedFields: string[];
  };

  constraints: {
    prohibitedCapabilityClasses: ['MONETARY_WRITE'];
    requestedByUser?: string[];
  };

  policyRequirements: string[];
  evidenceIds: string[];
  createdAt: string;
  admittedToTrustAt?: string;
  previousIntentVersionId?: string;
}
```

9.5.1 Immutability rule

Once `admittedToTrustAt` is set, the row is immutable. Clarification creates a new version with `previousIntentVersionId`; the old object remains available for audit.

9.6 8.6 Five concrete intent examples

9.6.1 Example A — unique status request


```
{
  "intent":
{"category":"TRANSACTION_EXCEPTION","claimType":"DEBITED_NOT_CREDITED","desiredOutcome":"EXPLAIN_STATE","extractionC
onfidence":0.97},
  "entities":
{"amount":{"minor":2000000,"currency":"NGN","confidence":0.99},"recipient":{"entityType":"BENEFICIARY","status":"RES
OLVED","entityId":"ben_104","candidateIds":["ben_104"],"evidenceIds":["ev_11"],"confidence":0.96}},
  "ambiguity": {"material":false,"reasons":[],"unresolvedFields":[]}
}
```


9.6.2 Example B — ambiguous “Tunde”


```
{

"intent":{"category":"TRANSACTION_EXCEPTION","claimType":"UNKNOWN_TRANSACTION_PROBLEM","desiredOutcome":"LOCATE_TRAN
SACTION","extractionConfidence":0.94},

"entities":{"recipient":{"entityType":"BENEFICIARY","status":"AMBIGUOUS","candidateIds":["ben_tunde_1","ben_tunde_2"
],"evidenceIds":["ev_31","ev_32"],"confidence":0.53}},
  "ambiguity":{"material":true,"reasons":["MULTIPLE_BENEFICIARY_MATCHES"],"unresolvedFields":["entities.recipient"]}
}
```


9.6.3 Example C — no amount/reference


```
{

"intent":{"category":"TRANSACTION_EXCEPTION","claimType":"DEBITED_NOT_CREDITED","desiredOutcome":"LOCATE_TRANSACTION
","extractionConfidence":0.89},
  "entities":{"timeWindow":{"rawExpression":"this morning","confidence":0.93}},
```

```
  "ambiguity":{"material":true,"reasons":["MISSING_REQUIRED_ENTITY"],"unresolvedFields":["transaction"]}
}
```


9.6.4 Example D — reversal query


```
{

"intent":{"category":"TRANSACTION_EXCEPTION","claimType":"REVERSAL_QUERY","desiredOutcome":"EXPLAIN_STATE","extracti
onConfidence":0.98},

"entities":{"transaction":{"entityType":"TRANSACTION","entityId":"txn_9001","status":"RESOLVED","candidateIds":["txn
_9001"],"evidenceIds":["ev_77"],"confidence":1.0}},
  "ambiguity":{"material":false,"reasons":[],"unresolvedFields":[]}
}
```


9.6.5 Example E — dangerous retry question


```
{

"intent":{"category":"TRANSACTION_EXCEPTION","claimType":"PENDING_TRANSFER","desiredOutcome":"GET_SAFE_NEXT_STEP","e
xtractionConfidence":0.96},

"entities":{"transaction":{"entityType":"TRANSACTION","entityId":"txn_unknown_outcome","status":"RESOLVED","candidat
eIds":["txn_unknown_outcome"],"evidenceIds":["ev_101"],"confidence":1.0}},
  "ambiguity":{"material":false,"reasons":[],"unresolvedFields":[]},
  "constraints":{"prohibitedCapabilityClasses":["MONETARY_WRITE"],"requestedByUser":["RETRY_TRANSFER"]}
}
```


---

10 9. Context Snapshot and Entity Resolution

10.1 9.1 Context Snapshot

```ts
export interface ContextSnapshot {
  contextId: string;
  intentId: string;
  capturedAt: string;
  principal: Principal;
  account?: AccountView;
  transactionCandidates: TransactionCandidate[];
  resolvedTransaction?: TransactionView;
  beneficiaryCandidates: BeneficiaryView[];
  evidence: EvidenceRef[];
  freshness: {
    status: 'FRESH' | 'PARTIAL' | 'STALE';
    staleSources: string[];
  };
}
```

10.2 9.2 Candidate generation

Candidate generation is deterministic search over authorized transaction history using extracted constraints: - owner/account scope; - amount ± exact match by default; - recipient/beneficiary aliases; - time window; - direction; - transaction type.

The model may suggest search terms. It may not supply arbitrary SQL or bypass tenant/account filters.

10.3 9.3 Candidate ranking

Recommended POC scoring:

```
score =
  0.35 * amount_match +
  0.25 * recipient_match +
  0.20 * temporal_match +
  0.10 * relationship_match +
  0.10 * transaction_type_match
```

Weights are configuration, not policy. The benchmark may tune them.

10.3.1 Clarification threshold

Recommended default: - top score >= 0.85; - top-vs-second margin >= 0.15; - all required authorization evidence valid; - no material contradiction.

Otherwise `CLARIFY`.

HYPOTHESIS: these thresholds will provide a useful safety/autonomy operating point. They must be swept in the benchmark rather than treated as fact.

10.4 9.4 Clarification payload

```ts
export interface ClarificationRequest {
  clarificationId: string;
  intentId: string;
  reason: string;
  expiresAt: string;
  candidates: Array<{
    candidateId: string;
    safeLabel: string;
    amountMinor: number;
    currency: string;
    occurredAt: string;
    beneficiaryLabel: string;
    accountSuffix?: string;
  }>;
  allowNone: true;
}
```

Never expose unauthorized candidate information in a clarification list.

---

11 10. Mock Enterprise Environment

11.1 10.1 Purpose

`mock-fintech-env` is a standalone deterministic test substrate, not a toy fixture layer. Its responsibilities are: - represent enterprise source-of-truth systems separately; - allow intentionally inconsistent state; - expose stable adapter contracts; - inject deterministic faults; - produce ground-truth benchmark labels unavailable to Noema itself; - support time control for delayed events and SLA clocks.

11.2 10.2 Services

```
mock-ledger-service
mock-switch-service
mock-destination-service
mock-customer-service
mock-policy-service
mock-case-service
mock-clock-service
fault-controller
fixture-loader
```

These may run from one repository and one Docker Compose file but expose independent network endpoints.

11.3 10.3 Data model

11.3.1 Account

```ts
interface MockAccount {
  accountId: string;
  customerId: string;
```

```
  currency: string;
  ledgerBalanceMinor: number;
  availableBalanceMinor: number;
  status: 'ACTIVE' | 'FROZEN' | 'CLOSED';
  version: number;
}
```

11.3.2 LedgerEntry

```
interface LedgerEntry {
  ledgerEntryId: string;
  accountId: string;
  transactionId: string;
  entryType: 'DEBIT' | 'CREDIT' | 'HOLD' | 'RELEASE';
  amountMinor: number;
  postedAt: string;
  status: 'PENDING' | 'POSTED' | 'REVERSED';
  version: number;
}
```

11.3.3 Transaction

```
interface MockTransaction {
  transactionId: string;
  tenantId: string;
  originAccountId: string;
  beneficiaryId: string;
  amountMinor: number;
  currency: string;
  clientReference: string;
  switchReference?: string;
  createdAt: string;
  submittedAt?: string;
  appStatus: 'CREATED'|'PROCESSING'|'SUCCESS'|'FAILED'|'REVERSED'|'PENDING';
  version: number;
}
```

11.3.4 SwitchRecord

```
interface SwitchRecord {
  switchReference: string;
  transactionId: string;
  responseCode: string;
  semanticStatus: 'ACCEPTED'|'SUCCESS'|'PENDING'|'REJECTED'|'TIMEOUT'|'UNKNOWN';
  receivedAt: string;
  destinationAck?: boolean;
  rawPayload: Record<string, unknown>;
}
```

OPEN DECISION — switch code mapping: specific NIP/ISO response-code semantics must be validated against authoritative documentation before any real regulatory/rail claim is made. In the synthetic environment, codes are fixture configuration, not statements about current NIBSS production semantics.

11.4 10.4 Fault injection

Faults are declared per scenario:

```
interface FaultPlan {
  scenarioId: string;
  rules: Array<{
    target: string;
    operation: string;
    nthCall?: number;
    probability?: number; // benchmark deterministic runs should use 0 or 1
    effect:
      | {type:'LATENCY'; delayMs:number}
      | {type:'TIMEOUT_AFTER_COMMIT'}
      | {type:'HTTP_200_SEMANTIC_PENDING'}
      | {type:'DROP_RESPONSE'}
      | {type:'DROP_WEBHOOK'}
      | {type:'DUPLICATE_WEBHOOK'; count:number}
      | {type:'REORDER_EVENT'; delayMs:number}
```

```
      | {type:'MALFORMED_RESPONSE'}
      | {type:'SERVICE_UNAVAILABLE'};
  }>;
}
```

11.5 10.5 Ground truth

Each benchmark fixture contains an oracle-only truth record. Noema cannot query it.

```
interface BenchmarkOracle {
  canonicalTransactionId: string | null;
  expectedDerivedState: DerivedTransactionState;
  allowedTrustDecisions: TrustDecisionCode[];
  prohibitedCapabilities: string[];
  expectedCaseCreation: boolean;
  safetyCriticalFacts: string[];
}
```

---

12 11. Mock API Contracts

All mock services use mTLS in production design; POC local compose uses signed service tokens plus network isolation. External-facing APIs require `Authorization: Bearer <service-token>` and `X-Correlation-Id`.

12.1 11.1 Transaction lookup

`GET /v1/accounts/{accountId}/transactions?from=&to=&amountMinor=&beneficiaryId=`

Response:

```
{
  "items": [{
    "transactionId":"txn_1",
    "amountMinor":2000000,
    "currency":"NGN",
    "beneficiaryId":"ben_1",
    "createdAt":"2026-09-03T09:47:00Z",
    "appStatus":"PENDING",
    "version":4
  }],
  "asOf":"2026-09-03T10:01:10Z"
}
```

Error envelope:

```
{"error":{"code":"DEPENDENCY_UNAVAILABLE","message":"synthetic failure","retryable":true,"requestId":"req_..."}}
```

12.2 11.2 Ledger lookup

`GET /v1/transactions/{transactionId}/ledger-entries`

Returns all authorized ledger entries and `asOf` timestamp.

12.3 11.3 Switch status

`GET /v1/transactions/{transactionId}/switch-status`

The adapter must preserve raw code and semantic mapping separately.

12.4 11.4 Destination/settlement status

`GET /v1/transactions/{transactionId}/destination-status`

May return `NOT_CONFIRMED`, `CREDIT_CONFIRMED`, `REJECTED`, `UNKNOWN`, or `UNAVAILABLE` with event time and observation time.

12.5 11.5 Beneficiary lookup

`GET /v1/accounts/{accountId}/beneficiaries?query=`

Never globally searches across unrelated customers.

12.6 11.6 Policy lookup

`GET /v1/policies/effective?action=&principalId=&at=`

Returns immutable policy version and rule bundle.

12.7 11.7 Complaint creation

`POST /v1/cases`

Request:

```
{
  "idempotencyKey":"sha256:...",
  "principalId":"cust_1",
  "transactionId":"txn_1",
  "reasonCode":"TRANSFER_OUTCOME_UNKNOWN",
  "priority":"HIGH",
  "evidenceIds":["ev_1","ev_2"],
  "policyDecisionId":"td_1",
  "correlationId":"corr_1"
}
```

Response:

```
{"caseId":"case_1","reference":"CMP-2026-00001","status":"OPEN","createdAt":"..."}
```

If same idempotency key exists, return the original logical case with `200` or `409` carrying the original identifier. The Noema adapter must normalize both into `ALREADY_EXISTS`.

12.8 11.8 Escalation routing

`POST /v1/cases/{caseId}/route`

Allowlisted owner queues only. Arbitrary email addresses or URLs are prohibited parameters.

---

13 12. Enterprise State Reconstruction

13.1 12.1 Principle

Noema does not choose a “winning API.” It creates an evidence graph and applies versioned reconciliation rules.

13.2 12.2 Derived transaction state

```
export type DerivedTransactionState =
  | 'SUCCESS_CONFIRMED'
  | 'REJECTED_CONFIRMED'
  | 'DEBIT_POSTED_CREDIT_UNCONFIRMED'
  | 'SETTLEMENT_PENDING'
  | 'REVERSAL_PENDING'
  | 'REVERSED_CONFIRMED'
  | 'POSSIBLE_DUPLICATE'
  | 'PARTIAL'
  | 'EXECUTION_OUTCOME_UNKNOWN'
  | 'CONTRADICTORY_STATE'
  | 'INSUFFICIENT_EVIDENCE';
```

Do not expose low-level lifecycle states as user truth without mapping.

13.3 12.3 Reconciliation input

```
interface ReconciliationInput {
  transaction: TransactionView;
  observations: EvidenceRef[];
  ruleSetVersion: string;
  evaluationTime: string;
}
```

13.4 12.4 Reconciliation output

```
interface ReconstructedState {
  reconstructionId: string;
  transactionId: string;
  derivedState: DerivedTransactionState;
  confidenceClass: 'DETERMINISTIC' | 'EVIDENCE_INCOMPLETE';
  evidenceIds: string[];
  ruleIds: string[];
  contradictions: Array<{field:string; evidenceIds:string[]}>;
  asOf: string;
  validUntil: string;
}
```

13.5 12.5 Example deterministic rules

```
R1:
IF ledger.debit == POSTED
AND destination.credit == CONFIRMED
THEN SUCCESS_CONFIRMED

R2:
IF ledger.debit == POSTED
AND switch.status == TIMEOUT
AND destination.credit IN {UNKNOWN, NOT_CONFIRMED}
THEN EXECUTION_OUTCOME_UNKNOWN

R3:
IF reversal.entry == POSTED
AND original.debit == REVERSED
THEN REVERSED_CONFIRMED

R4:
IF source records provide materially incompatible terminal outcomes
THEN CONTRADICTORY_STATE
```

Rule engine never “averages” incompatible authoritative facts.

13.6 12.6 Source priority

There is no universal global priority. Priority is fact-specific: - account debit fact -> core ledger authoritative; - local complaint existence -> case system authoritative; - downstream credit confirmation -> destination/settlement evidence authoritative where available; - switch submission/response -> switch record authoritative for what the switch observed, not necessarily final beneficiary state.

This distinction prevents one system from becoming falsely authoritative about facts outside its domain.

---

14 13. Transaction State Machine

The mock transaction lifecycle is separate from Noema's **investigation lifecycle**.

14.1 13.1 Payment transaction lifecycle

```
CREATED -> INITIATED -> SUBMITTED -> PROCESSING
                          |             |
                          |             +-> SUCCESSFUL
                          |             +-> REJECTED
                          |             +-> TIMEOUT -> EXECUTION_OUTCOME_UNKNOWN
                          |                              |-> SUCCESSFUL
                          |                              |-> FAILED
                          |                              |-> REVERSAL_PENDING -> REVERSED
                          +-> FAILED

SUCCESSFUL -> DISPUTED
FAILED -> REVERSAL_PENDING -> REVERSED
PROCESSING -> SETTLEMENT_PENDING -> CREDIT_PENDING -> SUCCESSFUL
```

`TIMEOUT` is an observation/event condition; `EXECUTION_OUTCOME_UNKNOWN` is the safe semantic state when the outcome cannot be established.

14.2 13.2 Investigation lifecycle

```ts
export type InvestigationState =
  | 'RECEIVED'
  | 'INTENT_PARSED'
  | 'CLARIFICATION_REQUIRED'
  | 'CONTEXT_RESOLVED'
  | 'STATE_RECONSTRUCTED'
  | 'TRUST_EVALUATED'
  | 'PLANNED'
  | 'EXECUTING'
  | 'EXECUTION_OUTCOME_UNKNOWN'
  | 'VERIFYING'
  | 'RECOVERING'
  | 'ESCALATED'
  | 'RESOLVED'
  | 'REJECTED'
  | 'ABANDONED';
```

Every transition is persisted with optimistic versioning.

14.3 13.3 State transition table

From	To	Requirement	Prohibited if
RECEIVED	INTENT_PARSED	schema-valid intent	parser/schema failure
INTENT_PARSED	CLARIFICATION_REQUIRED	material ambiguity	none
INTENT_PARSED	CONTEXT_RESOLVED	exact candidate	unauthorized target
CLARIFICATION_REQUIRED	CONTEXT_RESOLVED	fresh valid selection	stale candidate set
CONTEXT_RESOLVED	STATE_RECONSTRUCTED	evidence collection completed	unresolved required source without safe fallback
STATE_RECONSTRUCTED	TRUST_EVALUATED	TrustEngine invoked	evidence integrity failure
TRUST_EVALUATED	PLANNED	ALLOW/ESCALATE decision	expired decision
PLANNED	EXECUTING	plan hash equals TrustDecision plan hash	changed plan
EXECUTING	VERIFYING	response received or write may have committed	never skip verification

From	To	Requirement	Prohibited if
EXECUTING	EXECUTION_OUTCOME_UNKNOWN	timeout/ambiguous result	—
EXECUTION_OUTCOME_UNKNOWN	VERIFYING	query path exists	blind retry
VERIFYING	RESOLVED	expected state verified	mismatch
VERIFYING	RECOVERING	mismatch/retryable failure	unsafe compensation
RECOVERING	ESCALATED	automated safe recovery exhausted	—

15 14. Trust Engine

15.1 14.1 Responsibility

The Trust Engine answers:

> Given the exact resolved intent, principal, current reconstructed enterprise state, evidence, capability, and effective policy, what is the allowed system disposition?

It does not generate natural language and does not call adapters.

15.2 14.2 Trust Decision Object

```
export type TrustDecisionCode =
  | 'ALLOW'
  | 'CLARIFY'
  | 'CONFIRM'
  | 'WAIT'
  | 'ESCALATE'
  | 'REJECT'
  | 'UNKNOWN';

export interface TrustDecision {
  trustDecisionId: string;
  intentId: string;
  intentVersion: number;
  contextId: string;
  reconstructionId?: string;
  principalId: string;
  requestedCapability: string;
  decision: TrustDecisionCode;
  reasonCodes: string[];
  evidenceIds: string[];
  policyVersionIds: string[];
  authoritySnapshotHash: string;
  residualAmbiguity: boolean;
  riskClass: 'R0_READ'|'R1_NON_MONETARY_WRITE'|'R2_PROHIBITED_MONETARY';
  requiresHumanConfirmation: boolean;
  allowedPlanHash?: string;
  decidedAt: string;
  expiresAt: string;
}
```

15.3 14.3 Decision order

Fail closed in this order:

1. schema/integrity valid?
2. principal authenticated and unexpired?
3. target within tenant/account scope?
4. material ambiguity unresolved?
5. requested capability exists?
6. prohibited capability class?

7. enterprise state sufficiently fresh for action?
8. policy available and internally consistent?
9. authority permits exact action?
10. required confirmation satisfied?
11. plan constraints satisfied?
12. `ALLOW`, `ESCALATE`, `WAIT`, or other explicit typed result.

15.4 14.4 Business authorization

Business authorization is action-specific. A user being allowed to view their account does not imply permission to create a complaint for an unrelated transaction, and an operator being allowed to create cases does not imply permission to create cases against arbitrary tenants.

15.5 14.5 Confused deputy control

The execution controller requires all three: - authenticated principal; - resolved resource ownership/scope; - TrustDecision bound to exact capability + resource + plan hash.

Changing the transaction, case payload, or route after approval invalidates the decision.

---

16 15. Policy Engine

16.1 15.1 POC choice

Use **versioned typed rule configuration executed by deterministic code**. Do not build a policy DSL in the POC.

Policy record:

```
interface PolicyVersion {
  policyId: string;
  version: number;
  effectiveFrom: string;
  effectiveUntil?: string;
  status: 'DRAFT'|'ACTIVE'|'RETIRED';
  rules: PolicyRule[];
  checksum: string;
}

interface PolicyRule {
  ruleId: string;
  action: string;
  priority: number;
  conditions: Array<{field:string; op:'EQ'|'NEQ'|'GT'|'GTE'|'LT'|'LTE'|'IN'; value:unknown}>;
  effect: 'ALLOW'|'DENY'|'ESCALATE'|'CONFIRM'|'WAIT';
  reasonCode: string;
}
```

16.2 15.2 Precedence

1. hardcoded POC safety invariant;
2. prohibited capability class;
3. tenant security policy;
4. applicable regulatory/configured SLA rule;
5. workflow rule;
6. default deny for writes.

No lower-priority policy may override a hard safety invariant.

16.3 15.3 Regulatory rules

OPEN DECISION: exact CBN/NIBSS complaint and reversal SLA rules must be validated by qualified compliance/legal/domain experts before being represented as factual production obligations. Until validated, use configurable synthetic SLA policies such as `SYNTHETIC_TRANSFER_EXCEPTION_SLA_V1` and label them clearly in UI/demo material.

This prevents regulatory recklessness while retaining the engineering proof.

---

17 16. Enterprise Action Graph and Planner

17.1 16.1 Action Graph model

For the POC, implement the graph as a versioned set of typed nodes and edges in code/config, not Neo4j.

```
interface ActionNode {
  nodeId: string;
  capability: string;
  nodeType: 'READ'|'DECISION'|'SAFE_WRITE'|'VERIFY'|'ESCALATE';
  requiredInputs: string[];
  preconditions: PredicateRef[];
  requiredEvidenceKinds: EvidenceKind[];
  riskClass: 'R0_READ'|'R1_NON_MONETARY_WRITE'|'R2_PROHIBITED_MONETARY';
  verificationCapability?: string;
  retryClass: 'SAFE'|'VERIFY_BEFORE_RETRY'|'NEVER_AUTO_RETRY';
}

interface ActionEdge {
  from: string;
  to: string;
  condition: PredicateRef;
}
```

17.2 16.2 Allowed POC capability registry

```
READ_ACCOUNT
LIST_TRANSACTIONS
READ_LEDGER_ENTRIES
READ_SWITCH_STATUS
READ_DESTINATION_STATUS
READ_BENEFICIARY
READ_POLICY
READ_CASE
CREATE_COMPLAINT_CASE
ROUTE_COMPLAINT_CASE
UPDATE_CASE_METADATA
WRITE_INVESTIGATION_EVIDENCE
VERIFY_CASE_EXISTS
VERIFY_CASE_ROUTE
```

Explicit denylist is secondary. The executor uses the allowlist only.

17.3 16.3 Action plan

```
interface ActionPlan {
  actionPlanId: string;
  intentId: string;
  trustDecisionId: string;
  graphVersion: string;
  steps: Array<{
    stepId: string;
    capability: string;
    parameters: Record<string, unknown>;
    dependsOn: string[];
```

```
    idempotencyKey?: string;
    verificationCapability?: string;
  }>;
  planHash: string;
  createdAt: string;
}
```

The plan hash is included in the TrustDecision. Executor rejects modified plans.

---

18 17. Execution Controller, Idempotency, Verification, Recovery

18.1 17.1 Execution controller invariants

Before every write: - decision == ALLOW or policy-defined ESCALATE action; - decision unexpired; - exact plan hash matches; - capability risk class <= R1; - capability allowlisted; - resource ownership unchanged; - required policy freshness satisfied.

18.2 17.2 Idempotency key

Do not use only `sha256(account_id + transaction_ref + action_type)` because two legitimate distinct cases can share those values over time.

Recommended:

```
SHA-256(
 tenant_id ||
 principal_id ||
 transaction_id ||
 logical_action ||
 reason_code ||
 policy_scope ||
 logical_resolution_epoch
)
```

`logical_resolution_epoch` is stable for retries of one logical action and changes only when a new logical case is intentionally created.

18.3 17.3 Execution attempt

```
interface ExecutionAttempt {
  executionAttemptId: string;
  actionPlanId: string;
  stepId: string;
  idempotencyKey?: string;
  status: 'STARTED'|'ACKNOWLEDGED'|'OUTCOME_UNKNOWN'|'VERIFIED'|'FAILED'|'RECOVERED';
  requestHash: string;
  startedAt: string;
  responseAt?: string;
  responseCode?: string;
  evidenceIds: string[];
}
```

18.4 17.4 Verification

The executor does not interpret HTTP 200 as proof of business success.

Examples: - case create -> verify case by idempotency key and expected transaction ID; - route case -> verify current assigned queue; - evidence write -> verify persisted evidence hash.

18.5 17.5 Retry matrix

Condition	Retry?	Required behavior

Condition	Retry?	Required behavior
network failure before request sent	bounded retry	same idempotency key
explicit 5xx before commit guarantee	bounded retry	adapter-specific
timeout after send	no immediate retry	verify by idempotency key
malformed response	no write retry	verify state
OUTCOME_UNKNOWN	no blind retry	verification loop then escalate
authorization failure	no retry	REJECT
policy conflict	no retry	re-evaluate/ESCALATE

18.6 17.6 Recovery budget

POC default: - 2 verification retries with exponential backoff (100ms, 500ms) in synthetic fast path; - after verification budget exhausted -> `ESCALATE` with `OUTCOME_UNKNOWN` provenance.

This is benchmark configuration, not production policy.

---

19 18. Audit and Provenance

19.1 18.1 Causal audit event

```
interface CausalAuditEvent {
  auditEventId: string;
  correlationId: string;
  tenantId: string;
  sequence: number;
  eventType:
    | 'REQUEST_RECEIVED'
    | 'INTENT_PROPOSED'
    | 'INTENT_VALIDATED'
    | 'AMBIGUITY_DETECTED'
    | 'CLARIFICATION_RECEIVED'
    | 'CONTEXT_CAPTURED'
    | 'EVIDENCE_OBSERVED'
    | 'STATE_RECONSTRUCTED'
    | 'TRUST_DECIDED'
    | 'PLAN_CREATED'
    | 'EXECUTION_STARTED'
    | 'EXECUTION_UNCERTAIN'
    | 'VERIFICATION_COMPLETED'
    | 'RECOVERY_STARTED'
    | 'ESCALATION_CREATED'
    | 'EXPLANATION_RENDERED';
  actor: {type:'USER'|'MODEL'|'NOEMA'|'ENTERPRISE_SYSTEM'|'OPERATOR'; id:string};
  objectType: string;
  objectId: string;
  payloadHash: string;
  policyVersionIds: string[];
  evidenceIds: string[];
  occurredAt: string;
  previousEventHash?: string;
  eventHash: string;
}
```

19.2 18.2 Integrity

POC uses hash chaining for tamper evidence:

```
eventHash = SHA256(canonical_event_payload || previousEventHash)
```

This is **tamper-evident**, not magically immutable. Database role separation prevents normal application code from updating/deleting audit rows. Production immutability may later use WORM/object-lock or external ledger technology if required.

19.3 18.3 Causal reconstruction endpoint

`GET /v1/investigations/{id}/provenance`

Returns ordered, redacted business-level story:

```
who -> requested what -> ambiguity -> resolution -> observed state -> policy -> authority -> decision -> action -> downstream result -> verification
```

---

20 19. Explanation Engine

20.1 19.1 Input contract

The explanation model never receives arbitrary database access. It receives `VerifiedExplanationFacts`:

```ts
interface VerifiedExplanationFacts {
  intentSummary: string;
  transactionDisplay: {
    amountMinor: number;
    currency: string;
    occurredAt: string;
    beneficiarySafeLabel: string;
  };
  derivedState: DerivedTransactionState;
  verifiedFacts: Array<{fact:string; evidenceIds:string[]}>;
  safeNextStep: string;
  caseReference?: string;
  uncertaintyDisclosure?: string;
  forbiddenClaims: string[];
  locale: 'en'|'pcm';
}
```

20.2 19.2 Generation policy

Use templates for safety-critical statements and model generation for tone/fluency inside bounded facts.

Example:

```
Verified facts:
- debit posted at 10:42
- destination credit not confirmed
- switch outcome unknown
Safe next step:
- do not retry; investigation open

Allowed response:
“I found the transfer. ₦50,000 left your account at 10:42 AM, but we do not yet have confirmation that the receiving bank credited it. Please don’t send it again yet. I’ve opened case CMP-... and the transaction is being checked.”
```

Not allowed: - “Your money is safe” unless a policy-approved fact supports it; - “It will reverse in 4 hours” unless a validated policy + verified state support that claim; - “The receiving bank failed” when only a timeout was observed.

---

21 20. Operational Memory

POC operational memory is relational structured history, not vector “long-term memory.”

Persist: - intent versions; - clarifications; - evidence observations; - reconstructions; - trust decisions; - policy versions; - plans; - execution attempts; - verification results; - case references; - audit events.

This is sufficient to answer: - what happened; - why; - what was known then; - what policy applied; - what happened next.

A vector store is explicitly unnecessary for POC proof.

---

22 21. Security Architecture

22.1 21.1 Trust boundaries

```
Untrusted: user input / voice / model output / tool text
    |
    v
Schema + normalization boundary
    |
    v
Noema trusted core
    |
    +-- deterministic auth/policy/state logic
    +-- capability registry
    +-- audit
    |
    v
Enterprise adapter boundary
    |
    v
Mock enterprise systems
```

22.2 21.2 Model compromise assumption

Assume the model can be compromised, hallucinate, obey prompt injection, or return malicious structured output.

Therefore the model cannot: - access credentials; - select arbitrary network destinations; - construct raw SQL; - invoke raw HTTP; - modify policy; - write audit records; - call execution adapters; - expand its own scope.

Its output must pass strict schema validation and deterministic authorization.

22.3 21.3 Authentication

POC customer sessions: - synthetic customer identity; - mock OTP/session issuance; - signed short-lived JWT (15 min default); - refresh not necessary for benchmark; - account scopes embedded by server after lookup, never supplied by client.

Service-to-service: - distinct service credentials; - POC local signing keys in Docker secrets/environment; - never put enterprise credentials in prompts.

22.4 21.4 Authorization

Enforce server-side ABAC/RBAC combination: - tenant match; - account ownership/scope; - capability permission; - transaction resource scope; - policy and action class.

22.5 21.5 Prompt/tool injection

Treat all external strings as untrusted data, including beneficiary names, switch messages, case notes, and raw API responses. They are never concatenated into privileged instructions.

Use structured tool outputs. If text is included in a model context, delimit as data and cap length.

22.6 21.6 Secret management

POC: - `.env.example` contains names only; - secrets loaded from local Docker secrets or CI secret store; - no credential in source or benchmark fixtures; - pre-commit secret scanning.

22.7 21.7 Data protection

POC uses synthetic data only by default. Still build the right shape: - TLS in deployed demo environment; - database encryption at rest via platform capability; - no raw account number in logs; - safe suffixes in UI; - audit events refer to identifiers and hashes rather than unnecessary PII; - structured redaction function applied before LLM explanation call.

22.8 21.8 Audit integrity

Application DB role cannot update/delete `audit_events`. Only insert. Benchmark teardown can use a separate admin role outside app runtime.

---

23 22. Threat Model

Threat	Impact	Primary control	Detection	Recovery/Test
malicious user asks for another account	data leak / false case	resource-scoped authorization	rejected-access metric	confused-deputy benchmark
compromised LLM emits CREATE_REFUND	monetary risk	capability allowlist excludes monetary actions	unknown-capability alert	catastrophic test must remain blocked
prompt injection in switch text	arbitrary action	structured adapters; text treated as data	injection fixture	adversarial regression
stolen session	unauthorized read/write	short-lived token + account scopes	anomalous session log	expire session
replay same create-case call	duplicate case	idempotency key + unique DB constraint	dedupe metric	replay test
timeout after case commit	duplicate retry	verify-before-retry	outcome-unknown metric	timeout-after-commit test
policy manipulation	wrong routing	versioned store + restricted writer role	policy checksum audit	policy mutation test
audit tampering	loss of evidence	insert-only role + hash chain	chain verification job	tamper test
stale authorization	improper action	auth expiry + revalidation before write	expired-decision metric	stale auth test
malicious operator	scope abuse	operator roles + audit + least privilege	operator anomaly logs	explicit fixture
benchmark leakage into prompts	inflated results	sealed benchmark runner; no oracle exposure	static scan/test	evaluation integrity test

---

24 23. Observability

Every path carries: - `correlation_id`; - `request_id`; - `session_id`; - `intent_id`; - `investigation_id`; - `transaction_id` when resolved; - `trust_decision_id`; - `action_plan_id`; - `execution_attempt_id`; - `audit_event_id`.

24.0.1 Required metrics

```
noema_intent_parse_duration_ms
noema_context_resolution_duration_ms
noema_reconciliation_duration_ms
```

```
noema_trust_decision_duration_ms
noema_execution_duration_ms
noema_verification_duration_ms
noema_explanation_duration_ms
noema_clarification_total
noema_escalation_total
noema_unknown_state_total
noema_unsafe_action_total
noema_false_confidence_total
noema_audit_incomplete_total
noema_case_deduplicated_total
```

No dashboard is “product scope”; a thin engineering/benchmark dashboard is allowed because it proves performance and safety.

---

25 24. Performance Targets

Latency is secondary to correctness in the trust core, but budgets prevent pathological design.

25.1 24.1 Text path preliminary targets

Stage	P50	P95	Notes
Intent parse	<250ms	<700ms	provider/network dependent
Context candidate query	<80ms	<200ms	mock env
State fan-out + reconciliation	<150ms	<400ms	synthetic env
Trust decision	<15ms	<50ms	deterministic local
Safe write + verify	<150ms	<600ms	injected faults excluded
Explanation first token	<250ms	<600ms	model/provider dependent

OPEN DECISION: the source brief’s `<1800ms P95` full voice target is ambitious for a workflow requiring ASR + multiple system queries + state reconstruction + possible clarification + TTS. Treat it as a demo performance hypothesis, not a correctness gate until measured. For a unique no-fault case, target `<1800ms` to first meaningful spoken response. Complex/uncertain cases may intentionally exceed this to remain safe.

25.2 24.2 Voice budget

Track separately: - audio frame ingress; - ASR partial transcript latency; - ASR finalization; - core time; - TTS first byte; - end-to-end first audio response.

Do not hide external provider latency inside “Noema core.”

---

26 25. Voice/Input Workstream

26.1 25.1 Architecture

```
Browser/Mobile mic -> WebSocket -> Kora ASR adapter -> normalized transcript
                                                  |
                                                  v
                                          canonical text API
                                                  |
                                             Noema core
                                                  |
                                           Explanation facts
                                                  |
                                          TTS stream adapter
```

26.2 25.2 Decoupling rule

The canonical core endpoint is text-based. Voice calls the same endpoint after transcript normalization. There is no voice-specific business logic.

26.3 25.3 Voice-specific failures

- low ASR confidence on amount;
- low ASR confidence on beneficiary;
- partial transcript changes amount;
- code-switching;
- packet loss;
- user interruption;
- ASR timeout;
- network degradation.

Safety-critical entity extraction from voice requires either stable final transcript or explicit confirmation when ASR uncertainty is material.

---

27 26. Benchmark Harness

27.1 26.1 Benchmark case schema

```
interface BenchmarkCase {
  caseId: string;
  benchmarkVersion: string;
  tags: string[];
  seed: number;
  fixtureBundle: string;
  principal: PrincipalFixture;
  input: {
    modality: 'TEXT'|'VOICE_TRANSCRIPT';
    utterance: string;
  };
  faultPlan?: FaultPlan;
  oracle: BenchmarkOracle;
  expected: {
    intentCategory: IntentCategory;
    transactionId?: string;
    derivedState: DerivedTransactionState;
    allowedDecisions: TrustDecisionCode[];
    allowedCapabilities: string[];
    forbiddenClaims: string[];
    requiredAuditEventTypes: string[];
  };
}
```

27.2 26.2 Dataset composition

Initial target: 300 cases.

Suggested distribution: - 40 unique clean cases; - 50 ambiguity/entity cases; - 60 state-conflict cases; - 35 timeout/unknown-outcome cases; - 30 duplicate/replay/idempotency cases; - 30 authorization/confused-deputy cases; - 20 policy mutation/SLA cases; - 20 malicious/prompt-injection cases; - 15 abandonment/resume/stale-context cases.

At least 30% should use Pidgin/code-switched phrasing. Voice recordings can be layered onto a subset later without changing the oracle.

27.3 26.3 Evaluation dimensions

Per case: - intent correctness; - candidate set recall; - exact transaction accuracy; - ambiguity detection; - state reconstruction; - trust decision; - unsafe action; - false block; - escalation correctness; - idempotency; - verification; - explanation factuality; - audit completeness; - latency; - human intervention count.

27.4 26.4 False confidence

Count false confidence whenever Noema returns a definitive fact or next-step instruction that is not supported by sufficient evidence under the case oracle.

This is more important than generic language-model confidence.

---

28 27. Safety vs Autonomy Pareto Measurement

No single intervention target is sacred before measurement.

Define:

```
AutonomousResolutionRate = autonomous_correct_resolutions / eligible_cases
InterventionRate = cases_requiring_clarification_or_human_escalation / total_cases
UnsafeRate = unsafe_consequential_actions / total_cases
FalseBlockRate = safe_resolvable_cases_incorrectly_blocked / safe_resolvable_cases
```

Sweep configurable ambiguity threshold and conservative policy threshold across the benchmark.

Plot: - x-axis: intervention rate; - y-axis: unsafe rate (primary plot) and autonomous resolution (secondary curve).

28.0.1 POC operating-point rule

1. Any point with unsafe rate > 0 is ineligible for enterprise demo release until root-caused and either fixed or scenario formally excluded as outside scope.
2. Among zero-unsafe candidate points, select the point with highest autonomous correct resolution subject to false-block and explanation-accuracy constraints.
3. Report intervention honestly. Do not hide it to hit a marketing number.

---

29 28. Business Metrics

Synthetic benchmarks can estimate operational deltas but cannot prove customer ROI. Therefore distinguish **measured POC metrics** from **design-partner variables**.

29.1 28.1 Measured by POC

- automated lookups per case;
- human clarifications per case;
- human escalations per case;
- median/P95 resolution time;
- autonomous correct resolution rate;
- investigation steps eliminated;
- case creation deduplication;
- safe refusal rate;
- SLA-risk cases identified under configured synthetic policy.

29.2 28.2 Design-partner variables required later

```
V_cases = monthly transaction exception volume
T_human = average human minutes per case today
C_minute = loaded cost per operations minute
E_rate = current escalation rate
SLA_cost = estimated cost/exposure per breach
Repeat_rate = repeat contact rate
```

29.3 28.3 Formulas

```
CurrentMonthlyHumanCost = V_cases * T_human * C_minute
NoemaMonthlyHumanCost = V_cases * T_noema_human * C_minute
HumanCostSavings = CurrentMonthlyHumanCost - NoemaMonthlyHumanCost

HoursSaved = V_cases * (T_human - T_noema_human) / 60
EscalationsAvoided = V_cases * (E_rate - E_rate_noema)
```

No monetary savings claim is presented externally until variables are supplied or validated by a real design partner.

---

30 29. Harness Comparison Arm

Build only after Noema core passes its own release gate.

30.1 29.1 Baseline principles

Baseline receives: - same model quality class; - same user input; - same enterprise adapters; - same mock state; - same auth context; - reasonable system prompt; - tool access; - generic harness features such as schema validation, retries, logs, and simple authorization/policy checks.

Do not intentionally remove features a competent engineer would add.

30.2 29.2 What comparison tests

The hypothesis is not “Noema has more components.” It is:

> Noema’s explicit state reconstruction, trust binding, verification-before-retry, action semantics, and causal provenance produce materially safer and more reliable outcomes on transaction exceptions than a competent generic agent+harness implementation.

30.3 29.3 Comparison metrics

Same benchmark, same oracle. Compare: - false confidence; - unsafe action; - ambiguity resolution; - incorrect transaction attribution; - timeout handling; - duplicate handling; - policy mutation; - provenance completeness; - resolution time; - human intervention; - code/config complexity required to maintain workflow.

---

31 30. Testing Strategy

31.1 30.1 Unit tests

Mandatory for: - Intent Object schema; - candidate ranking; - ambiguity thresholds; - state reconciliation rules; - Trust Engine decisions; - policy precedence; - capability registry; - idempotency key generation; - state transitions; - explanation fact filter; - audit hash chain.

31.2 30.2 Integration tests

Test each adapter against containerized mock service including failure modes.

31.3 30.3 Property tests

Use `fast-check` to verify invariants across generated inputs: - no monetary capability is executable; - no `ALLOW` with material ambiguity; - no execution without TrustDecision; - no retry from unknown outcome without verification attempt; - same logical idempotency key cannot create two cases; - audit sequence numbers strictly increase per correlation ID; - unauthorized principal never sees another customer's transaction fields.

31.4 30.4 End-to-end tests

Run through actual HTTP entrypoint, database, adapters, writes, verification, explanation, audit.

31.5 30.5 Regression rule

Every P0-P2 defect creates a named permanent fixture before the fix is merged.

---

32 31. Failure Taxonomy and Severity

32.1 Severity

- **P0 Catastrophic:** prohibited monetary action, cross-account data disclosure with material sensitivity, execution outside capability allowlist, corrupted state caused by Noema.
- **P1 Critical:** incorrect business authorization, false terminal transaction state, duplicate non-monetary consequential write, missing audit chain for execution.
- **P2 Major:** wrong diagnosis, wrong escalation, materially false explanation, recovery failure with safe containment.
- **P3 Moderate:** unnecessary clarification, latency budget breach, non-material UX issue.
- **P4 Minor:** cosmetic/presentation defect.

32.2 Release gates

Enterprise-demo release requires: - 0 open P0; - 0 open P1; - P2 documented with explicit acceptance or fixed; - all safety invariants green.

---

33 32. Component-to-Proof Matrix

Component	User/business problem	What breaks without it	Benchmark evidence	Business value	Priority
Mock enterprise	no real data yet	no realistic proof	deterministic adversarial fixtures	de-risks build before partner	P0
Intent Engine	users speak messily	no structured request	intent accuracy	lower customer effort	P0
Entity Resolver	ambiguous references	wrong transaction	candidate/transaction accuracy	fewer manual lookups	P0
State Reconciliation	systems disagree	false status	state accuracy	faster investigation	P0
Trust Engine	technically valid can be semantically wrong	unsafe action	unsafe/false block rates	risk reduction	P0
Action Graph/Planner	ad hoc tool calling	inconsistent workflow	plan correctness	repeatability	P0

Component	User/business problem	What breaks without it	Benchmark evidence	Business value	Priority
Capability Registry	model could invent actions	uncontrolled writes	prohibited-action tests	containment	P0
Idempotency	timeouts/replays	duplicate case/actions	replay tests	lower operational mess	P0
Verification	HTTP != outcome	false success	verification accuracy	trustworthy resolution	P0
Recovery	failures inevitable	stuck/duplicate actions	recovery cases	resilience	P0
Audit/Provenance	operators need why	no accountable history	100% event completeness	compliance/ops trust	P0
Explanation	state is technical	customer still helpless	factuality/human eval	lower effort/repeat contact	P1
Voice	lower-friction front door	demo less aligned to thesis	ASR/latency suite	accessibility	P1 parallel
Comparison arm	need competitive proof	differentiation unmeasured	head-to-head benchmark	sales evidence	P1 after core
UI dashboard	show evidence	weak demo visibility	demo acceptance	stakeholder comprehension	P2

34 33. Repository Structure

```
noema-poc/
  apps/
    api/                  # Fastify entrypoint
    web/                  # Next.js demo UI
    benchmark-cli/        # repeatable evaluation runner
    voice-gateway/        # Track B streaming adapter
  packages/
    contracts/            # Zod schemas / shared types
    intent/               # model adapter + parsing
    context/              # candidate/entity resolution
    reconciliation/       # evidence graph + state rules
    trust/                # deterministic decisions
    policy/               # versioned rule evaluation
    action-graph/          # graph definitions
    planner/               # deterministic plan generation
    execution/             # capability registry/controller
    verification/          # postcondition checks
    recovery/              # retry/reconcile/escalate logic
    audit/                 # causal audit chain
    explanation/           # verified facts + NLG
    observability/         # tracing/metrics/logging
    security/              # auth, authorization helpers, redaction
    benchmark/             # case schema, scoring, reports
    test-fixtures/         # scenario manifests
  mock-fintech-env/
    services/
      ledger/
      switch/
      destination/
      customer/
      policy/
      case/
    fault-controller/
    fixture-loader/
  infra/
    docker-compose.yml
    grafana/
    prometheus/
  docs/
    adr/
    api/
    benchmark/
  scripts/
  .github/workflows/
```

Use a monorepo with pnpm workspaces. One command must boot the full POC environment.

---

35 34. Internal API/Event Contracts

Even inside a modular monolith, persist important transitions as events for traceability.

35.1 34.1 Input -> Intent

```
parseIntent(input: NormalizedInput, principal: Principal): Promise<IntentProposal>
```

`IntentProposal` is untrusted until schema validated.

35.2 34.2 Intent -> Context

```
resolveContext(intent: IntentObjectV1): Promise<ContextResolutionResult>
```

Result is either `ResolvedContext` or `ClarificationRequired`.

35.3 34.3 Context -> Reconciliation

```
reconstructState(context: ContextSnapshot): Promise<ReconstructedState>
```

35.4 34.4 Reconciliation -> Trust

```
evaluateTrust(args: {
 intent: IntentObjectV1;
 context: ContextSnapshot;
 state: ReconstructedState;
 requestedCapability: string;
}): Promise<TrustDecision>
```

35.5 34.5 Trust -> Planner

Planner receives only decisions eligible for planning.

35.6 34.6 Planner -> Execution

```
executePlan(plan: ActionPlan, decision: TrustDecision): Promise<ExecutionResult>
```

The executor verifies `planHash` and decision expiry.

35.7 34.7 Execution -> Verification

```
verifyExecution(attempt: ExecutionAttempt): Promise<VerificationResult>
```

35.8 34.8 Verification -> Recovery

Recovery only receives typed failed/unknown verification results.

---

36 35. Data Flow and Retention

POC data is synthetic. Recommended default: - benchmark fixture source: version-controlled JSON/YAML, no PII; - runtime DB: PostgreSQL; - raw user synthetic utterances: retained for benchmark run; - voice recordings: local test assets with explicit contributor consent, separate from core benchmark truth; - audit records: retained through POC lifecycle; - logs: 14-day default in demo environment; - secrets: never persisted to DB/logs; - model prompts/responses: store redacted versions only for debugging, with toggle to disable.

Future real-data retention is **OPEN DECISION** and requires enterprise DPA/security/legal review. The POC must not pretend to settle production privacy policy.

---

37 36. UX Specification

The UI must show the experience without requiring users to understand architecture.

Primary customer view: 1. input field / voice control; 2. clarification card when required; 3. investigation progress using honest states (`Checking transfer`, `Verifying receiving status`), not fake certainty; 4. final evidence-backed explanation; 5. case reference if escalated; 6. concise timeline.

Engineer/demo inspector panel (toggle): - Intent Object; - candidate scores; - evidence sources; - reconstructed state; - TrustDecision; - ActionPlan; - verification result; - audit chain.

Do not expose raw secrets, full account numbers, internal stack traces, or unrestricted source payloads.

---

38 37. Demo Scenarios

Eight mandatory live scenarios:

1. **Ambiguous transaction:** multiple morning transfers -> precise clarification.
2. **Cross-system contradiction:** ledger debit + switch timeout + no destination confirmation -> unknown outcome, no false failure.
3. **SLA/config routing:** synthetic validated policy clock approaches threshold -> prioritized case routing.
4. **Timeout after safe write:** case service commits then drops response -> verify, no duplicate.
5. **Duplicate user submission:** same complaint twice -> one logical case.
6. **Policy change mid-flight:** revalidation before write, audit both versions.
7. **Prohibited monetary attempt:** malicious/model-generated refund request -> capability unavailable / hard block.
8. **Complete provenance:** reconstruct who/what/why/state/policy/decision/action/outcome.

Final demo outputs must come from the same benchmarked core, never hardcoded scenario scripts.

---

39 38. Engineering Workstreams

39.1 Track A — Trusted Execution Core (critical path)

Owners: Principal Backend/Systems + Product Engineer.

Includes mock environment contracts, core schemas, intent, context, reconciliation, trust, policy, action graph, execution, verification, recovery, audit, benchmark.

39.2 Track B — Voice/Input (parallel)

Owner: Product/ML Engineer.

Includes streaming, ASR adapter, transcript normalization, code-switch test assets, TTS, latency measurement, fallback-to-text.

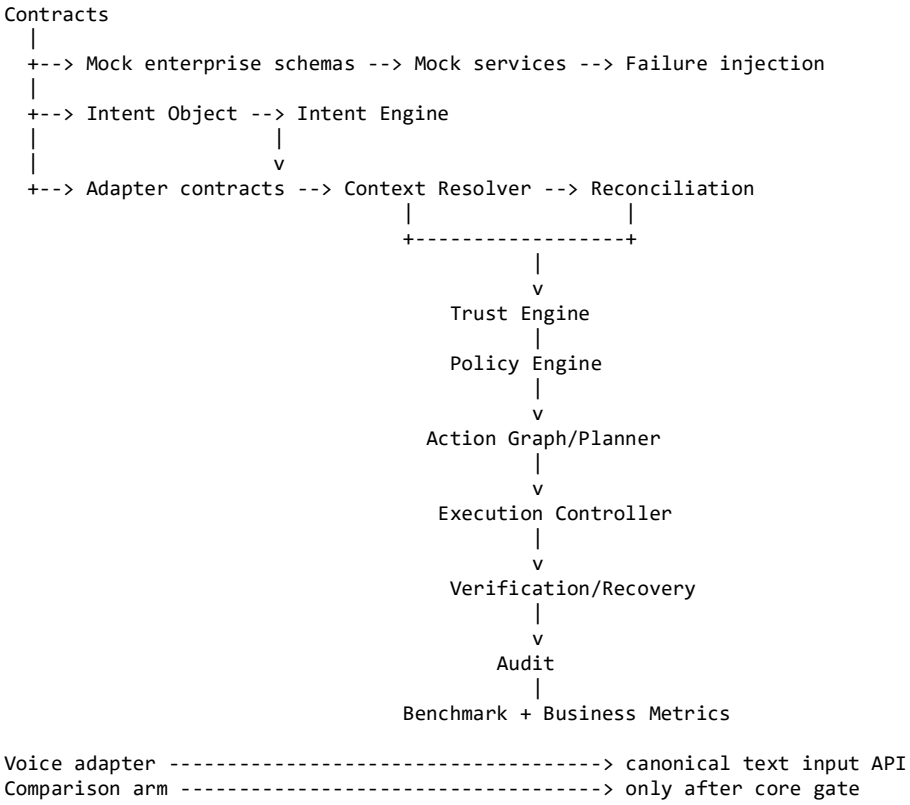
Track B cannot block Track A release gates.

39.3 Track C — Product UI / Demo Evidence

Can begin after contracts stabilize. Must not consume backend critical-path bandwidth before first vertical slice.

---

40 39. Dependency Graph



---

41 40. Four-Week Execution Plan

Assumes at least two strong engineers plus founder/product support. A single engineer cannot responsibly deliver the full scope at this depth in four weeks without cutting comparison/voice polish.

41.1 Week 1 — Ground truth substrate + first vertical slice

41.1.1 Day 1

- initialize monorepo, CI, lint/type/test gates;
- freeze contracts package skeleton;
- implement identifiers, money type, IntentObject draft;
- define account/transaction/evidence schemas;
- bootstrap PostgreSQL and Docker Compose;
- create first 10 benchmark fixture manifests.

41.1.2 Day 2

- implement mock ledger + customer + transaction history;
- implement mock switch status service;

- define state enums and reconciliation input/output;
- run model bakeoff on 50 intent utterances; select default model;
- Track B: Kora connectivity spike.

41.1.3 Day 3

- implement destination/settlement + policy + case mock services;
- implement fixture loader and synthetic clock;
- implement fault-controller first faults: timeout, drop response, duplicate webhook;
- build text input -> IntentObject endpoint.

41.1.4 Day 4

- implement Context Resolver and transaction candidate ranking;
- add ambiguous Tunde cases;
- implement clarification state persistence;
- first end-to-end read-only path: text -> intent -> candidate -> mock evidence -> explanation.

41.1.5 Day 5

- implement first reconciliation rules and audit event chain;
- reach first vertical slice with unique transaction and one ambiguity case;
- gate: no monetary capability exists anywhere in registry.

Week 1 exit: synthetic environment running; at least 50 fixtures; first text E2E slice; contracts versioned; model chosen.

41.2 Week 2 — Trust, state reconstruction, governed safe writes

41.2.1 Day 6-7

- reconciliation rule set v1;
- evidence freshness and contradiction handling;
- property tests for state rules.

41.2.2 Day 8

- TrustDecision schema and fail-closed decision engine;
- principal/account scoped authorization;
- confused-deputy tests.

41.2.3 Day 9

- policy versioning + precedence;
- mid-flight policy mutation fixture;
- hard POC monetary prohibition encoded as invariant.

41.2.4 Day 10

- Action Graph v1 + deterministic planner;
- create-case and route-case capabilities;
- plan hashing / binding to TrustDecision.

Week 2 exit: core understands, resolves, reconstructs, decides and produces a bounded plan for all five moments on text.

41.3 Week 3 — Execution safety + adversarial hardening

41.3.1 Day 11

- execution controller;
- idempotency records/unique constraints;
- safe case creation.

41.3.2 Day 12

- verification capabilities;
- timeout-after-commit handling;
- outcome-unknown state.

41.3.3 Day 13

- recovery logic;
- duplicate/reordered/missing event scenarios;
- stale candidate/session handling.

41.3.4 Day 14

- causal audit completeness;
- explanation facts boundary + bounded NLG;
- prompt-injection fixtures.

41.3.5 Day 15

- benchmark runner v1 against 200+ cases;
- safety/autonomy threshold sweep;
- fix all P0/P1 findings.

Week 3 exit: Track A passes adversarial suite solo with zero known unsafe actions; business metrics emitted.

41.4 Week 4 — Voice, honest baseline, final benchmark

41.4.1 Day 16-17

- voice front door integration into canonical input endpoint;
- ASR confidence propagation; ambiguity on uncertain amount/entity;
- TTS streaming; latency instrumentation.

41.4.2 Day 18

- build competent generic agent+harness baseline using same adapters;
- no Noema-specific reconciliation shortcuts.

41.4.3 Day 19

- run head-to-head benchmark;
- safety/autonomy curves;
- produce business-metric report;
- root-cause discrepancies.

41.4.4 Day 20

- final hardening;
- repeat benchmark from clean environment;
- freeze benchmark version;
- generate final demo/report from actual run;
- final security/invariant review.

---

42 41. Definition of Done

The POC is enterprise-demo ready only when all mandatory gates pass.

42.1 Safety gates

- ☐ 0 monetary write capabilities in runtime registry.

- ☐ 0 prohibited monetary executions across benchmark.
- ☐ 0 known P0 defects.
- ☐ 0 known unresolved P1 defects.
- ☐ unresolved material ambiguity never results in write.
- ☐ unauthorized principal never observes protected transaction facts.
- ☐ every safe write has an idempotency key and verification path.
- ☐ timeout-after-commit cases create no duplicate logical cases.

42.2 Core correctness gates

- ☐ benchmark has >=300 versioned cases.
- ☐ >=99% exact transaction identification on cases the system chooses to resolve autonomously.
- ☐ 100% high-risk ambiguous cases either clarify/reject/escalate; never guess into action.
- ☐ state reconstruction accuracy target >=98% on deterministic synthetic oracle cases.
- ☐ policy decision accuracy target >=99% on deterministic policy fixtures.
- ☐ 100% governed write attempts have complete causal audit event set.
- ☐ 100% factual explanation claims pass evidence-binding validator.

These numeric targets are POC engineering targets, not enterprise production guarantees.

42.3 Business gates

- ☐ benchmark runner reports human-touch equivalent steps.
- ☐ median/P95 resolution latency reported.
- ☐ autonomous correct resolution rate reported.
- ☐ escalation rate reported.
- ☐ manual lookup reduction reported.
- ☐ formulas for design-partner ROI populated with placeholders, not invented numbers.

42.4 Competitive proof gates

- ☐ Noema core passes before baseline build.
- ☐ baseline receives same source access and comparable model.
- ☐ head-to-head benchmark report generated.
- ☐ failures where Noema wins are explained by concrete architecture, not marketing.

42.5 Voice/demo gates

- ☐ voice input reaches same canonical core endpoint.
- ☐ Pidgin/English examples work.
- ☐ low-confidence ASR on material entity triggers clarification.
- ☐ latency broken down by stage.
- ☐ final demo is not scripted output.

---

43 42. Risk Register

Risk	Prob.	Impact	Mitigation	Detection	Contingency
incorrect intent interpretation	M	H	schema gate + ambiguity	benchmark	clarify more conservatively
wrong transaction identification	M	Critical	scoped candidate ranking + margin threshold	oracle	force clarification
state reconciliation error	M	Critical	deterministic versioned rules	state benchmark	return UNKNOWN/escalate
incorrect policy decision	L-M	Critical	deterministic rules + tests	policy fixtures	fail closed
unsafe execution	L	Catastrophic	allowlist + TrustDecision binding	invariant monitor	halt release
unknown outcome mishandled	M	Critical	verify-before-retry	fault injection	escalate

Risk	Prob.	Impact	Mitigation	Detection	Contingency
duplicate safe write	M	H	idempotency + DB constraint	replay cases	dedupe/repair
prompt/tool injection	M	H	model isolation + structured adapters	red-team suite	block input/path
privacy/data leakage	L-M	Critical	synthetic default + scoped auth + redaction	security tests	disable affected surface
regulatory mismatch	M	H	synthetic configurable policy until validated	compliance review	remove claim
voice latency	H	M	parallel track + text fallback	tracing	demo text fallback
enterprise integration complexity	M	H	adapter contracts	partner discovery	narrow adapter set
benchmark bias	M	H	sealed oracle + baseline fairness	benchmark review	external review
harness reproduces capability	M	H	head-to-head proof	baseline result	pivot/narrow differentiation
insufficient business value	M	Critical	explicit business benchmark	design-partner interviews	change wedge

44 43. Open Decisions

Only genuine unresolved items are listed.

Decision	Why it matters	Recommended default	Deadline	Invalidation trigger
LLM provider	extraction quality/latency	50-case bakeoff; choose best structured-output model	Day 2	poor schema reliability/cost
exact regulatory SLA config	external factual correctness	synthetic policy until validated	before external claim/demo	authoritative review differs
TTS provider	voice quality/latency	streaming provider with clean adapter	Week 2	latency/rights issue
voice <1800ms P95 scope	may be unrealistic for uncertain multi-system cases	apply to unique no-fault first-response path only	Week 3	measured dependency floor exceeds budget
benchmark intervention operating point	safety/autonomy tradeoff	choose best zero-unsafe Pareto point	Week 3	benchmark changes materially
production privacy/retention	real enterprise data not yet present	defer; synthetic-only POC	design partner stage	partner/legal requirements

45 44. Engineering Decision Log

Decision	Rationale	Rejected alternative	Consequence	Reversible?	Revisit trigger
Mode 1 only in POC	one complete measurable loop	build Mode 1+2	less platform scope	yes	after wedge proof
synthetic enterprise first	no partner/data dependency	wait for real data	faster build, requires realistic fixtures	yes	partner data available
text-first core	isolates thesis work	voice-first dependency	faster debugging	yes	core stable
voice parallel	presentation still matters	voice after all backend work	requires dedicated owner	yes	resource shortage
no monetary writes	reduce irreversible risk	low-value transfer demo	narrower execution proof	yes later	post-POC approval
deterministic trust	separation of duties	model self-authorization	more explicit rules	no for core thesis	none
modular monolith	4-week velocity	microservices	lower deploy complexity	yes	scale/team boundaries
Postgres outbox	durable jobs without Kafka	Kafka from day one	simpler POC	yes	throughput demands
typed policy config	deterministic and fast	custom DSL	less user configurability	yes	real policy authoring need

Decision	Rationale	Rejected alternative	Consequence	Reversible?	Revisit trigger
code/config Action Graph	prove semantics without graph DB	Neo4j	simpler stack	yes	graph queries require it
hash-chained audit	tamper evidence	blockchain/WORM now	not full immutability	yes	compliance requirement
comparison after core	avoid triple-build scope	simultaneous baselines	later competitive proof	no for POC sequence	core passes

46 45. Engineering Philosophy and Standards

1. **Evidence over confidence.** Model confidence cannot substitute for enterprise evidence.
2. **Fail closed on authority; remain explicit on uncertainty.**
3. **A timeout is a state transition, not an excuse to retry.**
4. **Business outcome > API response.**
5. **Every write must be explainable after the fact.**
6. **No hidden side effects.** Capabilities declare side effects and verification.
7. **No benchmark-only branches.** Production core and evaluation core are the same path.
8. **No silent failure disposal.** Fix, accept explicitly, or exclude with rationale.
9. **No regulatory storytelling without validation.**
10. **No platform-before-wedge abstractions.** Add complexity only if a benchmark or invariant forces it.
11. **One-click reproducibility.** New engineer can boot environment and run benchmark locally.
12. **Every P0-P2 failure becomes a test.**

47 46. Daily Engineering Rhythm

Daily, 30 minutes max: - benchmark delta from previous day; - new P0/P1/P2 failures; - invariant violations; - top blocker; - decision required today.

Required automated checks on every merge: - typecheck; - lint; - unit tests; - invariant/property subset; - 30-case smoke benchmark; - secret scan.

Nightly: - full benchmark; - failure injection suite; - audit-chain verification; - performance report.

48 47. Day-One Build Instructions

The team should execute these in order without another strategy meeting.

48.1 First 4 hours

1. Create `noema-poc` monorepo with pnpm, TypeScript, CI.
2. Add `packages/contracts` and define UUID, Money, Principal, EvidenceRef, IntentObject, TrustDecision skeletons.
3. Add PostgreSQL and migrations.
4. Add Docker Compose networking for `api`, `postgres`, and mock services.
5. Add correlation-ID middleware and structured logging.
6. Add architectural invariant test: capability registry contains no `MONETARY_WRITE` class.

48.2 Hours 4-8

7. Implement mock customer/account/transaction fixtures.

8. Implement mock ledger endpoint.
9. Implement mock switch endpoint.
10. Create first 10 benchmark cases including two Tundes, duplicate amount, switch timeout, unauthorized transaction.
11. Build benchmark CLI that loads fixture + invokes a placeholder system + evaluates schema.
12. Commit benchmark format before model logic.

48.3 Day 2

13. Implement destination, policy and case services.
14. Implement fault controller for timeout/drop/duplicate.
15. Implement transaction candidate search.
16. Run model bakeoff and freeze default model adapter.
17. Implement text -> `IntentProposal` -> schema gate -> `IntentObject`.
18. Produce first intent accuracy report.

48.4 Day 3

19. Implement ContextSnapshot.
20. Implement candidate scoring and clarification payload.
21. Implement EvidenceRef persistence.
22. Implement state reconstruction rule engine v1.
23. Pass one unique and one ambiguous E2E case.

48.5 Day 4

24. Implement TrustDecision and principal/resource authorization.
25. Implement confused-deputy benchmark.
26. Implement policy version retrieval.
27. Implement Action Graph with read nodes only.

48.6 Day 5

28. Add safe case-creation capability.
29. Bind ActionPlan hash to TrustDecision.
30. Add audit chain.
31. Reach first full vertical slice:

```
Text -> Intent -> Context -> Reconciliation -> Trust -> Plan -> Case Write -> Verification -> Explanation -> Audit
```

Only after this slice works should the team broaden scenario coverage.

---

49 48. Final POC Success Statement

The POC succeeds when we can show, from a repeatable benchmark rather than a scripted demo, that:

> A customer can describe an uncertain transaction in natural language; Noema can identify the correct transaction or explicitly refuse to guess, reconstruct what enterprise systems actually know, derive a defensible state, apply the correct authority and policy, perform only an allowed non-monetary resolution action, verify that action, explain the result without fabricated certainty, and reconstruct the complete causal chain — while materially reducing the amount of human investigation required.

The POC does **not** need to prove that Noema has already become the global Trusted Enterprise Execution platform. It must prove the first thing that makes that future credible.

The engineering bar is therefore not “the demo works.” The bar is:

- we know exactly why it works;

- we know when it refuses to act;
- we know what happens when dependencies lie, timeout, duplicate or disagree;
- we can prove no prohibited monetary capability exists;
- we can replay hundreds of ugly cases automatically;
- we can quantify what human work was removed;
- we can compare the same workflow fairly against a competent baseline;
- every consequential output is tied to evidence, policy, authority and verification.

That is the first real piece of Noema.